

O Ciclo ReAct e os Loops de Execução & Sandboxes, Permissões e o Controle de Execução — Um Recorte de Harness Engineering — Do Modelo ao Sistema Autônomo Confiável

Heverton Eduardo Peres

RESUMO

O ciclo de execução é a unidade fundamental do trabalho agêntico, e o controle do que o agente pode fazer determina a eficiência, a segurança e a auditabilidade da operação. Este artigo analisa o ciclo ReAct e os mecanismos de sandbox, permissões e controle de execução como componentes do harness, por meio de pesquisa qualitativa que integra literatura acadêmica, documentação técnica e literatura de segurança. Os resultados demonstram que a observação estruturada, a política de retentativa com backoff, limite e escalção, e o modelo de três zonas de execução com declaração prévia de política constituem a resposta de engenharia à tensão entre autonomia e controle. Conclui-se que a separação entre a intenção do modelo e a declaração do engenheiro é a defesa estrutural contra golpes de prompt e que a trilha de execução é requisito de primeira classe para a operação em escala.

Palavras-chave: Ciclo ReAct; loop de execução; sandbox; permissões de agente; harness engineering.

ABSTRACT

The execution loop is the fundamental unit of agentic work, and controlling what an agent may do determines the efficiency, security, and auditability of its operation. This article analyzes the ReAct loop and the mechanisms of sandboxing, permissions, and execution control as harness components, through qualitative research integrating academic literature, technical documentation, and security literature. The results demonstrate that structured observation, a retry policy with backoff, limits, and escalation, and the three-zone execution model with upfront policy declaration constitute the engineering answer to the tension between autonomy and control. It concludes that the separation between the model's intent and the engineer's declaration is the structural defense against prompt attacks, and that the execution trail is a first-class requirement for operation at scale.

Keywords: ReAct loop; execution loop; sandbox; agent permissions; harness engineering.

1 INTRODUÇÃO

1.1 Contextualização do Problema

O ciclo de execução é a unidade fundamental do trabalho agêntico. Diferentemente da chamada única a um modelo de linguagem, a operação de um agente em produção é um loop: raciocinar, agir, observar e repetir, com a observação realimentando o contexto da próxima decisão. O framework ReAct, formalizado por Yao e colaboradores, demonstrou que a alternância entre raciocínio e ação supera modos isolados em tarefas que exigem conhecimento externo e múltiplos passos (YAO et al., 2022). Essa constatação, combinada com a tese da OpenAI de que a qualidade do resultado é determinada pelo harness e não pelo modelo (OPENAI, 2026), coloca o loop de execução — e o controle do que o agente pode fazer — no centro da engenharia de sistemas agênticos.

A relevância do problema é quantificável: 57% das organizações pesquisadas já operam agentes em produção, e execuções únicas de longa duração — de até seis horas — são documentadas na operação real (LANGCHAIN, 2026; OPENAI, 2026). Nesse cenário, decisões de engenharia aparentemente menores — política de retentativa, tratamento de erro, permissões de execução, isolamento de ambiente — determinam se o agente conclui a tarefa, queima tokens em um loop infinito ou causa dano por executar fora do escopo (HU, 2026).

1.2 Objetivos

O objetivo geral deste artigo é analisar o ciclo ReAct e os mecanismos de sandbox, permissões e controle de execução como componentes do harness, respondendo à questão: como o harness garante que o agente execute tarefas longas de forma eficiente, segura e auditável? Os objetivos específicos são: (a) descrever a arquitetura do loop ReAct com observação estruturada; (b) analisar as políticas de retentativa com backoff, limite e escalação; (c) examinar os modelos de sandbox e zonas de execução; e (d) discutir a separação entre intenção do modelo e declaração de política do engenheiro.

1.3 Justificativa

A justificativa decorre do custo da ausência de controle. O relatório da LangChain indica que a observabilidade e o controle de execução são prioridades declaradas de 89% das organizações que operam agentes (LANGCHAIN, 2026). No campo da segurança, vulnerabilidades documentadas em agentes de codificação — incluindo a enganação de prompts de aprovação humana

e o envenenamento de ferramentas MCP — demonstram que a execução sem isolamento e sem política é a principal superfície de ataque (UTESVSKY, 2025; RED HAT, 2025). A OWASP situa a confiança excessiva em saídas do modelo e o controle inadequado entre os riscos críticos de aplicações de LLM (OWASP, 2025).

1.4 Delimitação

O estudo delimita-se ao loop de execução e ao controle de execução de agentes de engenharia de software. Não aborda a comparação de eficiência entre modelos de linguagem, nem a avaliação de benchmarks de desempenho, que pertencem a trabalhos futuros.

2 METODOLOGIA

2.1 Natureza da Pesquisa

A pesquisa é qualitativa e exploratória, combinando análise bibliográfica e análise de arquitetura de software. O objeto de análise é o ciclo de execução de agentes, estudado por meio de: (a) literatura acadêmica sobre raciocínio e ação em modelos de linguagem; (b) documentação técnica de harnesses de agentes em produção; e (c) literatura de segurança sobre controle de execução.

2.2 Coleta de Dados

O corpus foi coletado em três eixos. O primeiro eixo, acadêmico, inclui o paper fundacional do ReAct (YAO et al., 2022) e a proposta de código como harness (NING et al., 2026). O segundo eixo, técnico, inclui a documentação da OpenAI sobre harness engineering (OPENAI, 2026), a anatomia do harness de agentes da LangChain (TRIVEDY, 2026), as decisões de design arquitetural em harnesses (HU, 2026) e a análise da Databricks sobre AI agent harness (DATABRICKS, 2026). O terceiro eixo, de segurança, inclui o Top 10 da OWASP para LLMs (OWASP, 2025), a análise de riscos do Model Context Protocol (RED HAT, 2025), as vulnerabilidades documentadas por Utesvsky (UTESVSKY, 2025) e a análise de segurança de agentes autônomos de codificação (LASSO SECURITY, 2025).

2.3 Critérios de Inclusão

Foram incluídas fontes que (a) descrevem a arquitetura do loop de execução ou do controle de permissões; (b) documentam falhas, vulnerabilidades ou lições de operação de agentes; e (c) possuem URL verificável e autoria ou organização identificável. Fontes sem conteúdo técnico foram descartadas.

2.4 Procedimento de Análise

A análise seguiu três etapas. Na primeira, o corpus foi organizado em um dossiê de pesquisa com hierarquia de fontes (A, B e C). Na segunda, os conceitos foram mapeados para os dois eixos do estudo: o motor (loop ReAct, observação estruturada, política de retry) e o isolamento (sandbox, zonas de execução, permissões). Na terceira, procedeu-se à análise comparativa das abordagens, avaliando como cada decisão de design afeta a eficiência, a segurança e a auditabilidade da execução (HU, 2026; NING et al., 2026).

2.5 Limitações

A pesquisa é documental; não foram executados experimentos controlados com agentes em ambientes reais. A rápida evolução do campo sugere que parte das fontes de 2025 e 2026 pode ser atualizada em curto prazo, mitigado pela triangulação entre eixos independentes.

3 RESULTADOS E DISCUSSÃO

3.1 A Arquitetura do Loop ReAct

A análise confirma que o loop ReAct é o padrão canônico de execução agêntica. Em cada iteração, o agente recebe o contexto atual (instrução mais histórico de ações e observações), decide o próximo passo — raciocinar ou invocar uma ferramenta — e o harness executa a ação, devolvendo a observação estruturada como novo contexto (YAO et al., 2022). A decisão arquitetural central é que quem executa não é o modelo, mas o harness: o modelo propõe, o harness executa e devolve a realidade (OPENAI, 2026; BÖCKELER, 2026). Essa separação é o que permite observar, registrar e controlar cada ação.

Um achado relevante é a distinção entre observação estruturada e texto livre: o loop precisa diferenciar “a ferramenta respondeu 5” de “a ferramenta quebrou”. A literatura de design de harnesses enfatiza o contrato estruturado de saída — sucesso, dados e erro separados — como condição para que o loop decida com informação (HU, 2026; NING et al., 2026). Erros tratados como dados (registrar “503” como conteúdo da resposta) fazem o agente raciocinar sobre um erro como se fosse fato, perpetuando o desvio.

3.2 Política de Retentativa e Escalação

O tratamento de erro é o teste real do harness. A análise identificou três componentes da política madura de tentativa: backoff exponencial para falhas transitórias, limite de tentativas para nunca gastar sem teto e escalação para humano quando o limite é atingido (HU, 2026). A ausência de política produz os dois erros opostos documentados na operação real: retry

infinito em falha permanente, queimando tokens sem observação nova, e desistência precoce em falha transitória, entregando resultado incompleto (OPENAI, 2026). Execuções de até seis horas, documentadas em produção, só sobrevivem com política de erro robusta (OPENAI, 2026; LANGCHAIN, 2026).

3.3 Sandbox e Zonas de Execução

O segundo eixo do estudo — isolamento e permissões — revelou o modelo de três zonas como solução de referência: zona segura (execução livre), zona controlada (execução condicionada a políticas automáticas) e zona sensível (aprovação humana explícita e registrada) (HU, 2026; TRIVEDY, 2026). O achado mais importante é a separação entre intenção e declaração: a zona de uma ferramenta é declarada na configuração, antes da execução, e não decidida pela conversa. Essa separação impede o golpe de prompt — o adversário manipula a conversa, não a declaração — e torna o controle auditável (UTESVSKY, 2025; RED HAT, 2025).

3.4 Segurança da Execução

A literatura de segurança corrobora a necessidade do isolamento. O Model Context Protocol, adotado como padrão de conexão de ferramentas, apresenta riscos de servidores não confiáveis e clientes confusos que tornam a execução de ferramentas sem política um vetor de ataque (RED HAT, 2025; EMBRACE THE RED, 2025). Vulnerabilidades documentadas em cinco agentes de codificação demonstraram que prompts de aprovação podem ser enganados (UTESVSKY, 2025). A OWASP consolida o risco de confiança excessiva em saídas do modelo como um dos principais perigos de aplicações de LLM (OWASP, 2025). Esses achados sustentam que o controle de execução não é uma camada de conveniência, mas de segurança.

3.5 Implicações Práticas

As implicações práticas são diretas. Primeiro, toda execução de ferramenta deve passar por um ponto único controlado, com observação estruturada e registro (NING et al., 2026). Segundo, a política de retry deve combinar backoff, limite e escalação — nunca retry infinito (HU, 2026). Terceiro, a zona de cada ferramenta deve ser declarada na configuração, não decidida na conversa (UTESVSKY, 2025). Quarto, a trilha de execução deve permitir auditoria completa, com métricas de custo e sucesso — requisito declarado por 89% das organizações que operam agentes em produção (LANGCHAIN, 2026).

4 CONCLUSÃO

4.1 Síntese dos Achados

Este artigo respondeu à questão de pesquisa — como o harness garante execução longa, segura e auditável — demonstrando que a resposta reside no motor e no isolamento: o loop ReAct com observação estruturada (YAO et al., 2022), a política de retentativa com backoff, limite e escalação (HU, 2026), e o modelo de zonas com declaração prévia de política (TRIVEDY, 2026; UTESVSKY, 2025). A execução longa é viável apenas com política de erro robusta (OPENAI, 2026); a execução segura exige que a zona da ferramenta seja declarada antes, não negociada na conversa (UTESVSKY, 2025).

4.2 Contribuições

O artigo contribui com: (a) uma síntese da arquitetura do loop de execução agêntico, integrando o framework ReAct, o design de harnesses e a literatura de segurança; (b) a formalização do modelo de três zonas como resposta à tensão entre isolamento e utilidade; e (c) implicações práticas acionáveis para equipes que operam agentes em produção — ponto único de execução, política de retry limitada e declaração de zona por ferramenta.

4.3 Limitações e Trabalhos Futuros

As limitações decorrem da natureza documental da pesquisa. Trabalhos futuros devem: (a) medir empiricamente o impacto de cada componente da política de retry no custo e na taxa de sucesso; (b) avaliar modelos de sandbox em ambientes reais de produção; (c) investigar ataques emergentes a pontos de aprovação; e (d) desenvolver padrões declarativos de configuração de zonas, alinhados à proposta de código como harness executável e verificável (NING et al., 2026).

REFERÊNCIAS

AI-BOOST. *Awesome Harness Engineering*. 2026. Disponível em: <https://github.com/ai-boost/awesome-harness-engineering>. Acesso em: 09 ago. 2026.

ALEITHAN, Ali et al. *SWE-Bench+: Enhanced Coding Benchmark for LLMs*. 2024. Disponível em: <https://arxiv.org/abs/2410.06992>. Acesso em: 09 ago. 2026.

ANTHROPIC. *Introducing the Model Context Protocol*. 2024. Disponível em: <https://www.anthropic.com/news/model-context-protocol>. Acesso em: 09 ago. 2026.

BÖCKELER, Birgitta. *Harness engineering for coding agent users*. 2026. Disponível em: <https://martinfowler.com/articles/harness-engineering.html>. Acesso em: 09 ago. 2026.

DATABRICKS ENGINEERING. *What is an AI Agent Harness?* 2026. Disponível em: <https://www.databricks.com/blog/ai-harness>. Acesso em: 09 ago. 2026.

DORA. *Accelerate State of DevOps Report 2024*. 2024. Disponível em: <https://dora.dev/research/2024/dora-report/>. Acesso em: 09 ago. 2026.

EMBRACE THE RED. *MCP: Untrusted Servers and Confused Clients, Plus a Sneaky Exploit*. 2025. Disponível em: <https://embracethered.com/blog/posts/2025/model-context-protocol-security-risks-and-exploits/>. Acesso em: 09 ago. 2026.

GARTNER. 2025. *Gartner Predicts 40 Percent of Enterprise Apps Will Feature Task-Specific AI Agents by 2026*. Disponível em: <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>. Acesso em: 09 ago. 2026.

HU, W. *Architectural Design Decisions in AI Agent Harnesses*. 2026. Disponível em: <https://arxiv.org/html/2604.18071v1>. Acesso em: 09 ago. 2026.

JIM, Carlos et al. *SWE-bench: Can Language Models Resolve Real-world GitHub Issues?* 2023. Disponível em: <https://arxiv.org/abs/2310.06770>. Acesso em: 09 ago. 2026.

LANGCHAIN. *State of Agent Engineering 2026*. 2026. Disponível em: <https://www.langchain.com/state-of-agent-engineering>. Acesso em: 09 ago. 2026.

LASSO SECURITY (OXENBERG, O.; SUISA, E.). *Claude Code Security: Protect Autonomous Coding Agents*. 2025. Disponível em: <https://www.lasso.security/blog/claude-code-security>. Acesso em: 09 ago. 2026.

NING, X. et al. *Code as Agent Harness: Toward Executable, Verifiable, and Stateful Agent Systems*. 2026. Disponível em: <https://arxiv.org/html/2605.18747v1>. Acesso em: 09 ago. 2026.

OPENAI. *Harness engineering: leveraging Codex in an agent-first world*. 2026. Disponível em: <https://openai.com/index/harness-engineering/>. Acesso em: 09 ago. 2026.

OWASP FOUNDATION. *OWASP Top 10 for Large Language Model Applications*. 2025. Disponível em: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Acesso em: 09 ago. 2026.

RED HAT PRODUCT SECURITY (CANO GABARDA, F.). *Model Context Protocol (MCP): Understanding security risks and controls*. 2025. Disponível em: <https://www.redhat.com/en/blog/model-context-protocol-mcp-understanding-security-risks-and-controls>. Acesso em: 09 ago. 2026.

TRIVEDY, Vivek. *The Anatomy of an Agent Harness*. 2026. Disponível em: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>. Acesso em: 09 ago. 2026.

UTESVSKY, Roy. *SymJack: The approval prompt is lying to you*. 2025. Disponível em: <https://adversa.ai/blog/the-approval-prompt-is-lying-to-you-symlink-rce-in-five-ai-coding-agents-claude-code-cursor-antigravity-copilot-grok-build/>. Acesso em: 09 ago. 2026.

WIKIPEDIA. *Model Context Protocol*. 2024. Disponível em: https://en.wikipedia.org/wiki/Model_Context_Protocol. Acesso em: 09 ago. 2026.

YAO, Shunyu et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2022. Disponível em: <https://arxiv.org/abs/2210.03629>. Acesso em: 09 ago. 2026.